Contents

WoC-Bots: Implementation Specification

A build-ready specification for the WoC-Bots Reimagined capstone project

Stakeholder	Dr. Sean Grimes
Status	v1.1 — reviewed for release
Companion to	*WoC-Bots Reimagined* proposal; the WoC-Bots dissertation and publications

---

0. How to Read This Document

The dissertation and papers describe *what WoC-Bots are and why they work*. This document describes *how to build them*. It fills in the details a paper omits: initialization values, update rules, edge cases, orderings, and default parameters. Where the publications are ambiguous, this document makes a ruling so you don't have to guess.

Normative language:

- **MUST** — core to the method. If you change this, you are no longer replicating WoC-Bots.
- **SHOULD** — the reference design. Deviate if you have a reason, and document the deviation.
- **MAY** — genuinely open design space. Choose what fits your toolkit design.

You are replicating **the idea and the results**, not any particular codebase. Wisdom of crowds, many simple diverse agents, a spatial interaction arena, trust and certainty dynamics, and swarm-based opinion aggregation are the method. Variable names, thread models, and file layouts are not.

The only existing code you have access to is the `antevorta-db` module. It is referenced in §4 as a data source and as ground truth for dataset preparation. Everything else in this document is specified in prose, math, and pseudocode — implement it fresh, in Python, your way.

1. The Idea (One Page)

Traditional classifiers are monolithic: one model, trained on one complete view of the data. WoC-Bots replace the single expert with a **crowd**:

1. **Many simple agents.** Each agent wraps a very small classifier (an MLP with one to a few hidden layers) trained on a small *subset* of the available features (2–10 inputs). No agent sees the whole dataset. Duplicated feature sets across agents are allowed. This gives you a knowledge-diverse crowd — the precondition for wisdom-of-crowds effects (Page: collective error = average individual error − predictive diversity).
2. **Social interaction.** For each sample to classify, every agent forms an initial opinion from its own classifier, then agents are dropped into a 2D grid arena where they move randomly and interact pairwise when they land on the same cell. During an interaction, an agent shares its current prediction and its track record; the other agent updates its own **certainty** in its prediction — bolstered by agreement from trustworthy, confident, historically-correct agents; eroded by disagreement from the same. An agent whose certainty collapses below 0.5 flips its prediction. Interaction disperses information held by well-trained agents through the crowd.
3. **Opinion aggregation.** After the interaction period, the crowd's opinions are aggregated into one binary prediction. Two mechanisms, built in sequence:
   - **Trust-weighted voting** (Phase 1 of your project): each agent gets votes proportional to its historical accuracy and the trust other agents place in it.
   - **Honeybee swarm aggregation** (Phase 2): modeled on bee foraging — a rotating subset of agents "present" their opinion, the rest are recruited to presenters via fitness-proportionate selection, and the swarm iterates until a consensus threshold is reached. The threshold reached determines a **confidence label** for the prediction (Very High / High / Medium / Low), which is the method's most distinctive output: not just a prediction, but how much to believe it.

Why bother, when XGBoost exists? Because the architecture buys properties monolithic models don't have: samples with missing features are handled by simply excluding the affected agents; new features are absorbed by *adding agents* without retraining existing ones; the crowd distributes across machines; and (later) the "meta-swarm" lets an external model's prediction stand in for an agent's classifier, aggregating across institutions without sharing data. Raw accuracy parity with the state of the art is the table stakes; these properties are the contribution.

2. Vocabulary & Agent State

Every agent maintains the state below. Names are suggestions; semantics are normative. The symbols are the ones used in the equations in §6–§8 and in the dissertation.

State	Symbol	Range	Initialized to	Updated when
`current_prediction`	—	{0, 1}	Classifier output for the current sample	May flip during interaction

State	Symbol	Range	Initialized to	Updated when
`certainty`	a_{cert}	[0, 1]	(eval_accuracy + eval_precision) / 2	Every interaction; **reset per sample** (§6.6)
`trust_score`	a_{trust}	[0, 1]	eval_precision	Other agents modify it during interaction (§6.5)
`confidence`	a_{conf}	[0, 1]	Biased blend of eval metrics (§5.4)	Fixed after training
`prior_performance`	$a_{priorPerf}$	[0.7, 1.3]	1.0 (neutral)	As inference-time track record accrues (§6.7)
`prior_accuracy`	$a_{priorAcc}$	[0, 1]	eval_accuracy	Running accuracy over inference samples (§7.1)
`eval_accuracy` / `eval_precision` / `eval_recall`	—	[0, 1]	Measured on held-out eval data after training	Fixed after training
`features`	—	list	Assigned at agent creation	Fixed
interaction history	—	—	empty	Appended every interaction (§6.5)

Two performance notions coexist and are **not** the same thing — conflating them is a known implementation trap:

- **`prior_performance`** is a *multiplier* centered on 1.0 (range 0.7–1.3): "is this agent's advice historically better or worse than average?" It scales influence and trust updates.
- **`prior_accuracy`** is a *rate* in [0, 1]: "what fraction of this agent's predictions have been right?" It allocates votes.

---

3. System Lifecycle

The pipeline has four phases. Phases are sequential; within a phase, per-agent work is embarrassingly parallel. In Python you SHOULD just run phases sequentially in one process first — parallelism is an optimization, not part of the method.

```
PHASE 1 — TRAIN (once)
  for each agent:
      train its small MLP on its feature subset (training split)
      evaluate on held-out eval data -> eval_accuracy, eval_precision, eval_recall
      initialize certainty, trust_score, confidence, prior_performance
  prune agents with eval_accuracy < 0.50        # worse than a coin flip

PHASE 2..4 — repeated FOR EACH TEST SAMPLE:
  PHASE 2 — INFER
```

```
    each agent with complete features for this sample predicts 0/1 with its MLP
    agents missing a required feature value SIT OUT this sample entirely
    reset each participant's certainty to its training-time value
PHASE 3 — INTERACT (the arena, §6)
    initialize participants at random empty cells of a 2D grid
    for interaction_iters steps: move everyone, run pairwise interactions
PHASE 4 — AGGREGATE (§7 voting, or §8 swarm)
    produce the collective prediction (+ confidence label if using §8)
    update each participant's inference track record (prior_accuracy,
    prior_performance, per-partner interaction history correctness)
```

MUST: aggregate opinion is computed per test sample, after that sample's interaction period. There is no global "interact once, then classify everything" shortcut — the interaction dynamics depend on the agents' opinions *about the current sample*.

MUST: agents that were pruned in Phase 1, or that are missing features for the current sample, do not move, interact, or vote for that sample. This is the entire missing-data story, and it's a headline capability — make sure your arena and aggregation code handle a varying participant set per sample.

Degenerate-crowd ruling: if fewer than 3 agents can participate for a sample, skip the arena and swarm entirely — take a certainty-weighted vote of whoever is present (a crowd of one is just its own prediction), label the result Low confidence, and count the occurrence in your run manifest. A crowd that small has no wisdom to aggregate.

---

4. Data: Hollywood Movie Success

Phase-one benchmark. Binary question: **will this movie make more than 2× its production budget in revenue?** The 2× threshold approximates break-even after marketing costs, and splits the data nearly evenly (47.5% success / 52.5% failure) — so accuracy is a meaningful metric and a majority-class predictor scores ~52.5%.

4.1 Sources

Two public Kaggle datasets, joined:

- **TMDb 5000 Movie Dataset** (`tmdb_5000_movies.csv`, `tmdb_5000_credits.csv`) — budget, revenue, runtime, popularity, vote average/count, genres, cast/crew.
- **MovieLens 20M** (`links.csv`, `movies.csv`, `ratings.csv`, genome files) — per-user ratings (0–5 scale), genres, and the crucial `links.csv` mapping `movieId` ↔ `tmdbId` ↔ `imdbId`.

Two datasets are used (rather than TMDb alone) to densify vote information for older and less popular movies.

4.2 What `antevorta-db` gives you

The `antevorta-db` module (Kotlin) is the reference ETL: it ingests both raw datasets into SQLite and materializes the joined movie table with labels already computed. You can use it two ways:

- **Path A (reference/ground truth):** run its Hollywood ingestion to produce the SQLite database, then read that SQLite file from Python (`sqlite3` / `pandas.read_sql`). No Kotlin knowledge needed

beyond running a build; SQLite is language-neutral.

- **Path B (recommended for the toolkit):** re-implement the ETL in pandas as part of your Python package, following the normative rules in §4.3, and use Path A's output to *validate* yours (row counts, label balance, spot-check joins).

Useful landmarks in `antevorta-db` (citable, you have this code):

- Table names: `edu.antevortadb.configs.Finals` — `movies` (the joined TMDB movies table), `movielens_movies`, `links_table`, `individual_ratings`, `genome_tags`, `genome_scores`, `tmdb_credits`.
- Joined-table schema: `edu.antevortadb.dbInteraction.columnsAndKeys.TMDBMovies.columnNames()` — includes per-source columns (`tmdb_vote_average`, `tmdb_vote_count`, `movielens_vote_average`, `movielens_vote_count`, `tmdb_popularity`, `budget`, `revenue`, `runtime`, `tmdb_genres`, `movielens_genres`) **and precomputed label columns**: `made_more_than_2x_budget` (the binary target), plus `failure`, `mild_success`, `success`, `great_success`, `performance_class`, `missing_data`.
- Ingestion logic: `dbInteraction/dbcreator/hollywood/` (Facilitator/Pusher pairs — the `TMDBMoviesPusher` computes the performance-class labels at insert time).
- Query layer: `dbInteraction/dbSelector/hollywood/` (e.g., `MovieSelector`).
- Note: file paths in `configs/DBLocator.kt` and `configs/RawDataLocator.kt` are hardcoded for the original research machine (6-disk sharding). Expect to adjust or bypass them.

4.3 Normative preparation rules

1. **Join** ML and TMDb on movie identity using MovieLens `links.csv` (`tmdbId`). Keep only movies present in **both** datasets.
2. **Clean:** drop rows with impossible or missing-coded values — budget ≤ 0, revenue ≤ 0, or missing any feature used in the experiment. (In practice, zeros are the plague in the TMDb data — a $0 budget or $0 revenue means "unknown," not "free movie," and a zero revenue would otherwise silently label as failure.) Reference counts: 4,722 matched movies, 1,023 dropped as invalid, **3,699 usable** (≈740 test samples at an 80/20 split). Your counts should land close to these; investigate if they don't.
3. **Rating-scale alignment:** MovieLens ratings are 0–5; TMDb are 0–10. Multiply ML ratings by 2 before any combination.
4. **Per-movie ML aggregates:** MovieLens gives per-user ratings; aggregate to per-movie `ml_vote_count` (number of ratings) and `ml_vote_average` (mean rating, after ×2).
5. **Combined features:** `vote_count` = `tmdb_vote_count` + `ml_vote_count`; `vote_average` = count-weighted mean of the two per-source averages: `(tmdb_avg·tmdb_cnt + ml_avg·ml_cnt) / (tmdb_cnt + ml_cnt)`.
6. **Genres (optional feature):** use the *intersection* of the two datasets' genre lists for a movie; the reference work used the first two listed genres and found genre of limited value without more context. Treat as a stretch feature.
7. **Label:** `1` if `revenue > 2 × budget`, else `0`. (Matches the `made_more_than_2x_budget` column: note it is computed from a multi-class `performance_class` where class 0 = failure, so "mild success" — revenue between 1× and 2× budget — counts as `1` there. Verify against your own `revenue > 2·budget` rule and reconcile; document which definition you ship. The dissertation's stated definition, revenue > 2× budget, with the ~47.5/52.5 split, is the target.)
8. **Split:** 80% train / 20% test. Hold out an eval slice from the training side for the per-agent evaluation metrics (a 90/10 split of the training portion works; just don't let agents' eval data leak from the test set).
9. **Normalization:** min-max scale each feature to [0, 1] using statistics computed on the *training split*

only. `revenue` is never a feature — it defines the label. It MAY be used only for the sanity-check agent (§9.2).

Available features for agents: `budget`, `vote_count`, `vote_average`, `tmdb_popularity`, `runtime`, plus the per-source vote columns and (optionally) genres.

4.4 Quarter 2 preview: Airline Passenger Satisfaction

The second benchmark (don't build for it yet, but don't design against it): the Kaggle *Airline Passenger Satisfaction* dataset — **~120,000 samples, 22 input features** (demographics, flight distance/delays, and a battery of 1–5 service-satisfaction ratings: WiFi, boarding, seat comfort, food, legroom, baggage handling, check-in, etc.).

- **Label:** satisfied vs. not — passengers reported satisfied / neutral / dissatisfied; **neutral counts as dissatisfied** (no experiments were run with neutral counted as satisfied).
- **Structured missingness:** a `0` in a 1–5 satisfaction feature means "not applicable" — this is exactly the missing-data story of §3, arriving for free. An agent whose features are N/A for a sample sits out.
- **Scale:** 120k samples × per-sample arena runs is where a naive O(everything) implementation dies. This is why §11 recommends keeping the arena lean, and it is the natural motivation for vectorization/parallelism work in Q2.
- Published ballpark to aim for: mid-90s accuracy (~95%) with the swarm mechanism.

---

5. Agents

5.1 Feature assignment

- Every agent receives **1–4 highly correlated features shared with all agents** — in practice, `budget` is the anchor feature every agent gets. Correlation with the target was originally established with PCA run offline; you SHOULD reproduce this analysis once as a notebook-style experiment — rank features by point-biserial correlation with the label and/or first-component PCA loadings (scikit-learn), confirm `budget` and `vote_count` top the list — then hard-code the anchor set in experiment configs. The analysis is rerun per dataset, not per experiment.
- Remaining features are distributed randomly across agents. Duplicate feature sets across agents are **allowed** — neither limited nor encouraged.
- Agent count scales with the experiment (5 for small feature sets up to 26+ for the full feature set — see §9.3). More unique agents generally helped performance.

5.2 Classifier

Each agent's classifier MUST be small and shallow. Reference architecture:

- Input: the agent's 2–10 features.
- Hidden layers: `max(1, round(0.3 × input_size))` layers — so 2–3 features → 1 hidden layer; 10 features → 3. Keep it as shallow as possible while still learning.
- Hidden width: unconstrained by the method; **default 32 units per hidden layer**, a few dozen is plenty. MAY tune.
- Output: 2 nodes, softmax (or 1 sigmoid unit — equivalent for binary; pick one and be consistent about extracting P(class=1)).
- Optimizer: Adam, learning rate 0.001. Loss: cross-entropy.

- Epochs: 5–50. Accuracy saturates fast (see §9.1 targets); ~20 epochs was the reference sweet spot. Mini-batch size 32.

`scikit-learn`'s `MLPClassifier(hidden_layer_sizes=..., solver='adam', learning_rate_init=0.001, batch_size=32, max_iter=<epochs>)` is a fine base. A `LogisticRegression` agent variant MAY be offered as a config option — the architecture does not care what's inside an agent, and demonstrating that is on-theme.

5.3 Pruning

After training, evaluate each agent on held-out eval data. Agents with **eval_accuracy < 0.50** MUST be excluded from all subsequent phases. Log what was pruned — a config that prunes half its crowd is telling you the feature assignment is bad.

5.4 Confidence (fixed after training)

$$conf = w_{acc} \cdot acc + w_{prec} \cdot prec + w_{rec} \cdot rec, \quad \sum w = 1$$

The bias weights are per-problem configuration. For Hollywood, weight precision highest — reference weights **(0.3, 0.5, 0.2)**. (For a medical dataset you'd bias recall, e.g. (0.25, 0.25, 0.50) — the point of the knob.) `confidence` answers "how good was this agent at training time, weighted by what this problem cares about," and it does not change during inference.

---

6. The Interaction Arena

6.1 Geometry

- 2D grid of discrete cells. **Number of cells = 2 × number of participating agents**, as square as possible: `side = ceil(sqrt(2N))`. (The abstraction supports arbitrary rectangular compositions — rooms, floors — but the experiments use a plain square. Build the plain square; leave the interface open.)
- A cell holds at most **2 agents**. Two agents in a cell = an interaction. Three-agent interactions do not exist.

6.2 Initialization

Each participating agent is placed at a uniformly random **empty** cell (no sharing at init). Initialization strategy is a pluggable interface (`init()`) — random is the reference; clustering-similar-agents or maximal-spread are documented open experiments.

6.3 Movement

Pluggable interface (`move()`). Reference algorithm, per iteration, per agent:

1. Move one step in a uniformly random cardinal direction (N/S/E/W), staying in bounds. Moving onto a cell with one occupant is what triggers an interaction; moving onto a cell with two occupants is not allowed.

2. **Anti-clique rules:** two agents MUST NOT interact twice in a row, and MUST NOT interact more than twice within any 5-iteration window. An agent with no legal move is **teleported** to a random empty cell.
3. **Stirring:** occasionally teleport an agent at random even when it could move legally (reference: a small probability, ~5% per agent per iteration — this is a MAY-tune parameter) to keep information dispersing.

Movement is synchronous-in-rounds: all agents move once, then all interactions for the round resolve, then the next round begins. (Sequential processing within a round is fine — iterate agents in random order each round to avoid ordering artifacts.)

6.4 Interaction period length

$$interaction\_iters = \max(10,\ round(0.10 \times N))$$

where N = participating agents. The 0.10 factor balanced information dispersal against runtime; 0.25 was tested and bought nothing. The `max(10, ...)` floor is a ruling this document adds: the published formula was tuned on larger crowds, and 5-agent experiments still need a handful of rounds for opinions to circulate.

6.5 The interaction (certainty & trust updates)

During an interaction each agent exposes a read-only **public profile** to its partner: `current_prediction`, `certainty`, `confidence`, `trust_score`, `prior_performance`, and its feature list. Private internals (the classifier itself, raw training data, full history) are never shared — this locality is what later makes the meta-swarm's privacy argument work, so keep the boundary crisp in code (a `public_profile()` method or frozen dataclass, not "reach into the other agent").

When agents a and b share a cell, **both directions execute**: a receives from b, then b receives from a (using a's pre-update state for symmetry — compute both deltas, then apply; or accept the tiny asymmetry and document it. Reference behavior: compute-then-apply is cleaner; pick one).

Receiving side (a receiving from b):

```
acceptance      = 1.0 - a.certainty                      # open-mindedness
influence       = b.confidence * acceptance * (b.trust_score * b.certainty)
corrected       = influence * b.prior_performance        # scale by track record
if a.current_prediction != b.current_prediction:
    corrected   = -corrected                             # disagreement erodes
a.certainty     = clamp(a.certainty + corrected, 0.01, 0.99)
if a.certainty < 0.50:
    a.current_prediction = 1 - a.current_prediction      # flip
    a.certainty = 1.0 - a.certainty                      # believe the flip as much
                                                        # as you doubted the old view
```

The clamp is a ruling this document adds (the equations otherwise permit certainty to escape [0, 1] after a streak of agreements; 0.99 cap also keeps `acceptance` from pinning to exactly 0 so agents never become fully deaf).

Worked example: a (certainty 0.62, predicts 1) meets b (predicts 0, confidence 0.71, trust 0.78, certainty 0.80, prior_performance 1.1). acceptance = 0.38; influence = 0.71 × 0.38 × (0.78 × 0.80) = 0.168; corrected = 0.168 × 1.1 = 0.185, negated for disagreement → −0.185. a.certainty = 0.62 − 0.185 = 0.435 < 0.5 → a

flips to predict 0, certainty = 0.565. One confident, trustworthy dissenter can flip a lukewarm agent — that's intended.

Trust update (also during the interaction; each side may adjust the *other's* trust score): a consults its interaction history with b — the fraction of past interactions where b's advice (i.e., b's prediction at the time) turned out correct once ground truth was known, $b_{percCorrect}$.

$$b_{trust} \mathrel{+}= 0.05 \cdot (b_{percCorrect} \cdot b_{priorPerf}) \cdot doAgree$$

with $doAgree = +1$ if the two agents currently agree, -1 otherwise, and NO update if a has no prior history with b. Clamp trust to [0, 1]. The 0.05 factor caps any single interaction's trust movement at 5%.

MUST: interaction history is recorded per (agent, partner, sample): partner's prediction at interaction time, so that after ground truth arrives (§6.7) correctness can be back-filled. This history is also your primary analysis/debugging instrument — the dissertation tracked opinion flow through the crowd with it. Keep it queryable (a tidy DataFrame or SQLite table beats nested dicts).

Truthfulness (`truth()`) and interaction willingness (`shouldInteract()`) exist in the interface; the reference implementation is always-willing, always-truthful, and `updateTrust` as above. Deceptive/selective agents are a documented future experiment.

6.6 Per-sample reset semantics

MUST: at the start of each test sample, every participant's `certainty` resets to its training-time value and `current_prediction` comes fresh from its classifier. Certainty drift during an arena period reflects evidence *about one sample* and does not carry over. `trust_score`, `prior_performance`, `prior_accuracy`, and interaction history DO persist and accumulate across samples — they are the crowd's long-term social memory.

6.7 Ground-truth feedback

After the collective prediction for a sample is recorded, reveal the true label (test-set evaluation) and update: each participant's `prior_accuracy` (running fraction correct of its *own* end-of-arena predictions), `prior_performance`, and interaction-history correctness back-fill. For `prior_performance`, the ruling:

$$priorPerf = clamp(1.0 + 0.6 \times (running\_acc - 0.5),\ 0.7,\ 1.3)$$

so a coin-flip agent sits at the neutral 1.0, a perfect agent at 1.3, and the scale is linear in between. Anything monotone in running accuracy, neutral at 0.5, bounded to [0.7, 1.3], and documented is acceptable. Until an agent has ≥ 5 scored predictions, keep `prior_performance = 1.0` (cold start; don't let two lucky guesses mint a 1.3× influencer). In a deployment scenario labels arrive late or never; the system tolerates this — values simply stop updating.

---

7. Aggregation, Mechanism 1: Voting (build first)

Implement all three; they are each other's baselines and reproduce a published figure.

1. **UWM (Unweighted Mean Model):** every agent gets 100 votes for its predicted class. Majority wins.

2. **WVM (Weighted Voter Model):** votes = `round(100 × prior_accuracy)`. Before any track record exists, 50 votes.
3. **Trust-weighted (the contribution):**

$$votes_a = round\left(\frac{a_{priorAcc} + a_{trust}}{2} \times 100\right)$$

Collective prediction = class with the larger vote total; **ties go to class 1** (document this; with ~100-vote granularity ties are rare but real). Method 3 SHOULD consistently beat 1 and 2 — if it doesn't, your trust dynamics are broken (§10).

Optional vote-margin confidence tiers (a cheap precursor to §8's confidence output): report `max(votes_0, votes_1) / total_votes` and bucket it — ≥95% near-certain / 90–95% very high / 75–90% high / 65–75% medium / 52–65% low / <52% coin-flip. Useful for the report's confidence-calibration analysis even in Phase 1.

---

8. Aggregation, Mechanism 2: Honeybee Swarm (build second)

Bee-foraging-derived consensus. This replaces §7 as the aggregation step; training and arena interaction are unchanged. Its two claims, which your experiments should verify: (a) lower variance across runs/folds than WVM, (b) a meaningful confidence label per prediction.

8.1 One swarm round

```
def swarm_round(participants, rng):
    presenters = rng.sample(participants, k=max(1, round(0.20 * len(participants))))
    watchers   = [a for a in participants if a not in presenters]

    # fitness-proportionate (roulette-wheel) assignment of watchers to presenters
    for p in presenters:
        p.fitness = (p.prior_performance_norm + p.confidence + p.trust_score) / 3
        #  ^ use prior_performance rescaled to [0,1] here: (pp - 0.7) / 0.6,
        #    so the three terms share a scale            (ruling by this document)
    normalize fitness over presenters -> selection probabilities
    for w in watchers:
        w.assigned = roulette_pick(presenters)

   # dissenting strong watchers may re-roll, at most twice, no guarantee of improvement
    for w in watchers:
        moves = 0
        while (moves < 2 and w.current_prediction != w.assigned.current_prediction
               and w.prior_performance > w.assigned.prior_performance):
            w.assigned = roulette_pick(presenters)   # same wheel; may land worse
            moves += 1

    # each presenter's opinion now carries 1 + (# assigned watchers) votes
    tally votes per class over presenters
```

```
    agreement = max(class_votes) / total_votes       # weighted by watcher counts
    return agreement, majority_class                 # exact tie: agreement = 0.5,
                                                  # no threshold met, swarm continues
```

20% presenters mirrors the scout fraction in real colonies. The two-move cap prevents infinite reshuffling and deliberately leaves some watchers supporting presenters they disagree with — retained diversity, analogous to mutation in genetic algorithms.

8.2 The confidence ladder

```
round 1:            if agreement == 100%  -> predict, VERY HIGH confidence, stop
rounds 2..101:      if agreement >= 90%   -> predict, HIGH confidence, stop
rounds 102..151:    if agreement >= 75%   -> predict, MEDIUM confidence, stop
otherwise:          certainty-weighted vote of ALL agents (presenters + watchers):
                    score_c = sum of a.certainty for agents predicting class c;
                    larger score wins, exact tie -> class 1
                    -> predict, LOW confidence
```

Between rounds, **within each presenter's group**, every watcher interacts (per §6.5 rules, minus the spatial part) with each co-watcher and with the presenter — this is where opinions continue to shift during swarming. Then a fresh 20% of presenters is drawn and the next round runs.

MUST: "Very High" is achievable only on the very first vote, before any swarm-phase interaction — that purity is what bought 100% accuracy in that band on the reference dataset. Do not soften it.

The threshold values (100/90/75) and round budgets (1/100/50) were tuned on the breast cancer data with a goal of 100%-accurate Very High predictions; they are configuration, not constants of nature. Your toolkit SHOULD expose them, and auto-tuning them per dataset is a flagged open research question — a genuinely publishable extension.

Reference behavior to sanity-check against (breast cancer data, so expect different numbers on Hollywood — the *shape* is what matters): confidence bands ordered by accuracy (VH 100% > High 93.1% > Medium 82.3% > Low 64.7%); dropping the Low band raised overall accuracy from ~80% to 86.8% while retaining 78.9% of samples; variance across folds collapsed vs. WVM (accuracy σ^2 2.3 vs 24.5).

---

9. Evaluation Protocol & Reference Results

9.1 Protocol

- Report accuracy, precision, recall (and for the swarm: per-confidence-band accuracy and coverage).
- **Every reported number is a mean over 10 runs with different seeds, with standard deviation.** The method is stochastic (init, movement, presenter selection); single runs are not results. Where folds are used, keep folds fixed when comparing mechanisms.
- Hollywood uses a fixed 80/20 split + 10 seeded runs. From Q2 onward, prefer 5-fold cross-validation (the swarm-vs-WVM variance comparison is a fold-level analysis).
- Baselines, same data, same splits: a monolithic MLP over all features (Phase 1); XGBoost, random forest, logistic regression (Phase 2).
- **Baseline MLP spec (Phase 1):** all available features as inputs, a **single hidden layer** (width ~2× the input count; MAY tune), same optimizer settings as the agents (Adam 0.001, cross-entropy, batch 32),

trained 5 and 50 epochs, same normalization pipeline. It is the published comparison target — resist the urge to make it fancier; a tuned deep network is a Phase 2 baseline, not the Phase 1 reference.
- Seed everything through one `numpy.random.Generator` passed down from the experiment config. A results table that can't be regenerated from a config file + seed is not done.

9.2 Reference numbers to reproduce (Hollywood)

Individual agent MLPs after training (accuracy, 5 / 50 epochs):

Agent features	5 ep	50 ep
budget, revenue *(sanity check only)*	98%	100%
budget, vote_average, vote_count	77.2%	77.6%
budget, tmdb_popularity, vote_average, vote_count	75.4%	75.7%
budget, vote_count	75.7%	75.5%
budget, tmdb_popularity, tmdb_vote_average, tmdb_vote_count	72.8%	74.9%
budget, tmdb_vote_count, ml_vote_count	73%	73.4%
budget, ml_vote_average, ml_vote_count	62.2%	64.1%
budget, ml_vote_count	60.3%	61.9%
budget, tmdb_vote_average	60.9%	61.4%
budget, runtime	53.9%	56.4%

The budget+revenue agent MUST hit ~100% — it can see the answer. If it doesn't, your data pipeline is broken. If your other agents land within a few points of this table, your agent layer is calibrated.

Crowd-level (trust-weighted voting, 5 features — budget, vote_count, vote_average, runtime, popularity — five agents: four 2-feature agents each anchored on budget, one 5-feature agent): WoC-Bots optimum **76.3% at 20 epochs** vs. monolithic MLP optimum **76.8% at 40 epochs**. WoC-Bots reach their optimum in half the epochs; the MLP edges them out slightly at full training in this deliberately-matched configuration. With larger crowds (26 agents, mixed 2/3/4/5-feature agents), WoC-Bots beat the MLP in 5 of 6 feature configurations.

Robustness targets: removing `vote_count` costs WoC-Bots ~1.9% vs the MLP's ~4%; removing noisy `runtime` improves the MLP ~1.7% but WoC-Bots only ~0.3% (bad agents get marginalized rather than removed). Reproducing these two contrasts is a Phase 1 deliverable.

Match expectations: land within **~2–3 percentage points** of the crowd-level numbers and preserve every *ordering* (mechanism 3 > 2 > 1; the robustness contrasts). This is replication of stochastic published work from its description — direction and magnitude, not decimals.

9.3 Agent-count guidance

Reference configurations: 4–5 agents for 2–3 feature experiments; 11 agents at 4 features; 26 agents at 5 features (one 5-feature, five 4-feature, ten 3-feature, ten 2-feature). "Many, many agents" is the regime the

method wants; the toolkit should make `n_agents` and the feature-assignment policy pure configuration.

---

10. Pitfalls (Read Before Debugging)

Failure modes this method invites; most were learned the hard way.

1. **Certainty explosion.** Without the [0.01, 0.99] clamp, agreeable crowds saturate certainty and `acceptance` hits 0 — interactions become no-ops and the arena does nothing. If the arena phase never changes any opinion, check this first.
2. **Conflated performance scales.** `prior_performance` (multiplier ~1.0) inside influence math vs. `prior_accuracy` (rate ~0.7) in vote allocation. Swapping them quietly deflates or inflates influence by ~30%.
3. **Interaction ordering artifacts.** Fixed agent iteration order each round biases who flips first. Shuffle per round. (Prediction-market agents fail precisely because participation order swings outcomes — WoC-Bots' low variance across orderings is a *claim* your tests should verify, not assume.)
4. **Vote-formula off-by-scale.** `(priorAcc + trust)/2 × 100` — both terms in [0, 1]. Feeding a percentage into one slot gives one agent 5,000 votes and a very confusing accuracy chart.
5. **Test-set leakage into agent eval.** Agents' eval_accuracy (which seeds certainty, trust, pruning) MUST come from held-out *training-side* data, never the test split.
6. **Skipping the reset (§6.6).** Certainty carrying across samples looks fine for 50 samples, then agents ossify and accuracy decays over the test run. If accuracy drifts downward as inference proceeds, look here.
7. **Missing-participant crashes.** Arena sizing (2×N), interaction_iters (0.1×N), and the 20% presenter count all MUST be computed from *this sample's* participant count.
8. **Degenerate swarm rounds.** With 5 agents, 20% presenters = 1 presenter = instant 100% agreement = everything Very High confidence. Enforce `k ≥ 2` presenters when $N \geq 10$, and treat $N < 10$ swarm results as "voting fallback" territory (document the ruling). The swarm mechanism earns its keep at crowd sizes of dozens-plus.
9. **The budget+revenue agent contaminating real runs.** It exists to validate the pipeline. It sits out every reported experiment; its 100 votes and perfect trust would otherwise dominate every aggregation. (Even the published aggregation underweights a uniquely-superb agent — known limitation, don't "fix" it silently.)
10. **Trying to reproduce exact decimals.** Ten stochastic runs on a 740-sample test set have ~1.5-point standard errors. Chase orderings and bands (§9.2), not digits.

---

11. Python Toolkit Shape (Guidance, All MAY)

```
wocbots/
  data/       loaders + ETL (Hollywood first; the antevorta-db SQLite reader here too)
  agents/       Agent (state + lifecycle), classifier wrappers (sklearn MLP, logreg)
  arena/     Grid, InitPolicy, MovementPolicy  (protocols/ABCs; random + manhattan impls)
  interaction/  InteractionPolicy (certainty/trust math of §6.5), history store
  aggregation/  UWM, WVM, TrustWeighted, HoneybeeSwarm  (one interface, four impls)
  experiments/  config-driven runners (YAML/dataclass configs, seeds, output manifests)
  evaluation/   metrics, multi-run statistics, plots, baseline models
tests/          unit tests on §6.5/§7/§8 math with hand-computed cases (use §6.5's
```

```
worked example); an end-to-end test on a synthetic separable dataset
where the crowd MUST hit ~100%
```

The pluggable-policy interfaces, matching the published architecture (names are suggestions; the seams are not):

```python
class InitPolicy(Protocol):
    def place(self, agent, arena, rng) -> Cell: ...          # empty cell, in bounds

class MovementPolicy(Protocol):
    def move(self, agent, arena, rng) -> Cell: ...           # legal cell or teleport

class InteractionPolicy(Protocol):
    def should_interact(self, a, b) -> bool: ...             # reference: always True
    def truth(self, a, b) -> bool: ...                       # reference: always True
    def update_trust(self, a, b) -> None: ...                # §6.5 rule; may be no-op

class ScoringPolicy(Protocol):
    def update_prediction(self, agent, partner_profile) -> None: ...   # §6.5 math

class Aggregator(Protocol):
    def aggregate(self, participants, rng) -> Prediction: ...  # UWM/WVM/Trust/Swarm
```

- Pure-Python objects + numpy; no framework needed for the arena. Mesa is acceptable if the team knows it, but the grid + rounds loop is ~100 lines and owning it keeps the interaction rules exact.
- Every experiment = one config file; every result = config + seed + git SHA. That's the reproducibility story the original lacked, and it's a stated deliverable.
- Keep policies (init/movement/interaction/scoring/aggregation) as swappable interfaces — modularity here *is* the method's design claim, and it's what makes the later quarters (meta-swarm, new datasets) cheap.

---

12. Multi-Quarter Roadmap

Quarter 1 — Core replication (Hollywood). Data pipeline (§4) validated against antevorta-db output; agent layer (§5) hitting the §9.2 agent table; arena (§6); the three voting mechanisms (§7); monolithic-MLP baseline; reproduce the §9.2 crowd numbers, robustness contrasts, and mechanism ordering. *Exit criterion: the §9.2 reference results regenerate from configs.*

Quarter 2 — Swarm + comparison. Honeybee aggregation with confidence ladder (§8); airline dataset onboarding (§4.4); XGBoost/RF/logreg baselines; variance study (swarm vs WVM); confidence-calibration analysis; missing-data-tolerance experiments (progressively delete feature values, watch graceful degradation vs imputing baselines). *Exit criterion: swarm beats WVM on variance with comparable accuracy; confidence bands are monotone in accuracy on both datasets; the full comparison table (WoC-Bots vs 4 baselines × 2 datasets) regenerates from configs.*

Quarter 3+ — Distinctive capabilities. Incremental feature addition (train new agents on new features mid-stream, inject, no retraining of the crowd — measure lift); meta-swarm (an agent whose "classifier" is an external model's prediction — combine heterogeneous sources without data sharing); optional distribu-

tion story; auto-tuned confidence thresholds (§8.2); write the paper. *Exit criterion: at least one distinctive-capability experiment with a defensible, plotted result — that experiment is the paper's contribution.*

---

13. Symbol / Equation Cross-Reference

For reading alongside the dissertation (Chapter: *WoC Bots — Hollywood*, and Chapter: *Swarm-based Opinion Aggregation*):

Dissertation	This document
Eq. acceptance $a_{acceptance} = 1 - a_{certainty}$	§6.5 `acceptance`
Eq. influence / trustCertainty / correctedInfluence	§6.5 `influence`, `corrected`
Eq. certainty update + flip at 0.5 + reflection	§6.5 flip block
Eq. trust update (5% cap, percCorrect, doAgree)	§6.5 trust update
Eq. biasedVotes $((priorAcc + trust)/2) \cdot 100$	§7 mechanism 3
Eq. probability $(priorPerf + conf + trust)/3$	§8.1 fitness
interaction_iters $= 0.10 \cdot N$	§6.4 (with floor)
Arena spaces $= 2 \cdot N$	§6.1
Confidence categories VH/H/M/L	§8.2 ladder
confidence bias example $(0.25, 0.25, 0.50)$	§5.4 (Hollywood uses 0.3/0.5/0.2)

Where this document specifies something the publications do not (clamps, floors, tie breaks, prior_performance mapping, presenter minimums, reset semantics), it says so inline — those are rulings, made so the team builds one system instead of five interpretations. Disagreements with a ruling are welcome; raise them with the stakeholder rather than silently diverging.